

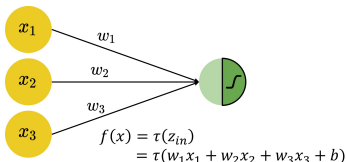
From a Neuron to a Network

The single neuron, the XOR problem, and the first hidden layer

This deck embeds the LMU *Introduction to Deep Learning* mechanics chunks in full (single neuron $\rightarrow$ XOR $\rightarrow$ single hidden layer), then adds where to *play* with it and how it connects to training.

Deep Learning

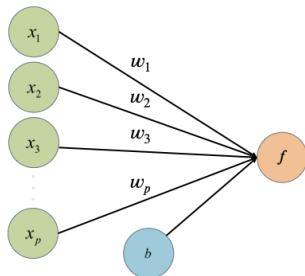
Single Neuron / Perceptron



Learning goals

- Graphical representation of a single neuron
- Affine transformations and non-linear activation functions
- Hypothesis spaces of a single neuron
- Typical loss functions

A SINGLE NEURON



Perceptron with **input features** $x_1, x_2, \dots, x_p$, **weights** $w_1, w_2, \dots, w_p$, **bias term** b , and **activation function** τ .

- The perceptron is a single artificial neuron and the basic computational unit of neural networks.
- It is a weighted sum of input values, transformed by τ :

$$f(\mathbf{x}) = \tau(w_1x_1 + \dots + w_px_p + b) = \tau(\mathbf{w}^T\mathbf{x} + b)$$

A SINGLE NEURON

Activation function τ : a single neuron represents different functions depending on the choice of activation function.

- The identity function gives us the simple **linear regression**:

$$f(\mathbf{x}) = \tau(\mathbf{w}^T \mathbf{x}) = \mathbf{w}^T \mathbf{x}$$

- The logistic function gives us the **logistic regression**:

$$f(\mathbf{x}) = \tau(\mathbf{w}^T \mathbf{x}) = \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x})}$$

A SINGLE NEURON

We consider a perceptron with 3-dimensional input, i.e.

$$f(\mathbf{x}) = \tau(w_1 x_1 + w_2 x_2 + w_3 x_3 + b).$$

- Input features $\mathbf{x}$ are represented by nodes in the “input layer”.

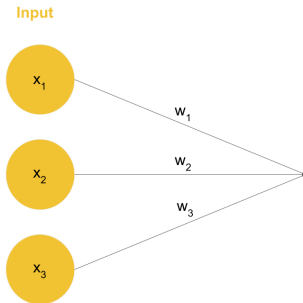
Input



- In general, a p -dimensional input vector $\mathbf{x}$ will be represented by p nodes in the input layer.

A SINGLE NEURON / 2

- Weights $\mathbf{w}$ are connected to edges from the input layer.



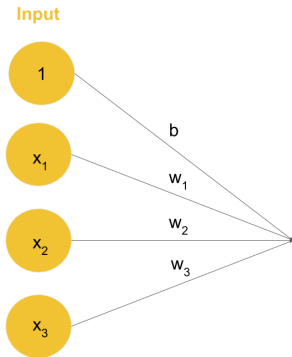
- The bias term b is implicit here. It is often not visualized as a separate node.

A SINGLE NEURON / 3

For an explicit graphical representation, we do a simple trick:

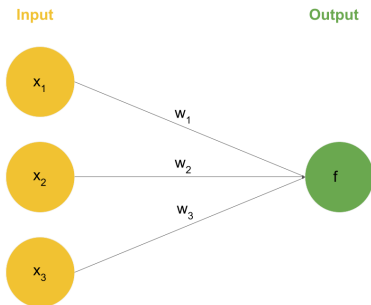
- Add a constant feature to the inputs $\tilde{\mathbf{x}} = (1, x_1, \dots, x_p)^T$
- and absorb the bias into the weight vector $\tilde{\mathbf{w}} = (b, w_1, \dots, w_p)$.

The graphical representation is then:



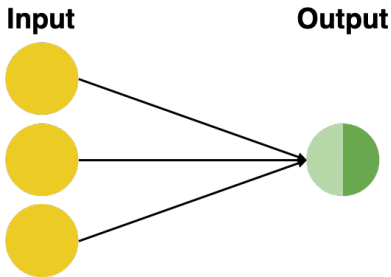
A SINGLE NEURON / 4

- The computation $\tau(w_1x_1 + w_2x_2 + w_3x_3 + b)$ is represented by the neuron in the “output layer”.



A SINGLE NEURON / 5

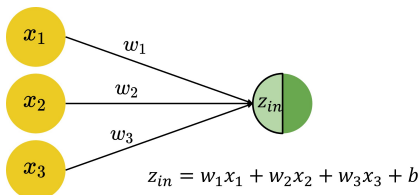
- You can picture the input vector being "fed" to neurons on the left followed by a sequence of computations performed from left to right. This is called a **forward pass**.



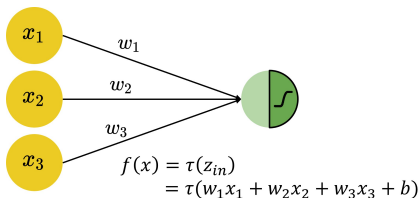
A SINGLE NEURON

A neuron performs a 2-step computation:

- 1 **Affine Transformation:** weighted sum of inputs plus bias.



- 2 **Non-linear Activation:** a non-linear transformation applied to the weighted sum.



A SINGLE NEURON: HYPOTHESIS SPACE

- The hypothesis space that is formed by single neuron is

$$\mathcal{H} = \left\{ f : \mathbb{R}^p \rightarrow \mathbb{R} \mid f(\mathbf{x}) = \tau \left(\sum_{j=1}^p w_j x_j + b \right), \mathbf{w} \in \mathbb{R}^p, b \in \mathbb{R} \right\}.$$

- If τ is the logistic sigmoid or identity function, $\mathcal{H}$ corresponds to the hypothesis space of logistic or linear regression, respectively.

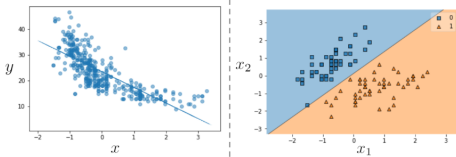


Figure: *Left:* A regression line learned by a single neuron. *Right:* A decision-boundary learned by a single neuron in a binary classification task.

A SINGLE NEURON: OPTIMIZATION

- To optimize this model, we minimize the empirical risk

$$\mathcal{R}_{\text{emp}} = \frac{1}{n} \sum_{i=1}^n L \left(y^{(i)}, f \left(\mathbf{x}^{(i)} \right) \right),$$

where $L(y, f(\mathbf{x}))$ is a loss function. It compares the network's predictions $f(\mathbf{x})$ to the ground truth y .

- For regression, we typically use the L2 loss (rarely L1):

$$L(y, f(\mathbf{x})) = \frac{1}{2}(y - f(\mathbf{x}))^2$$

- For binary classification, we typically apply the cross entropy loss (also known as Bernoulli loss):

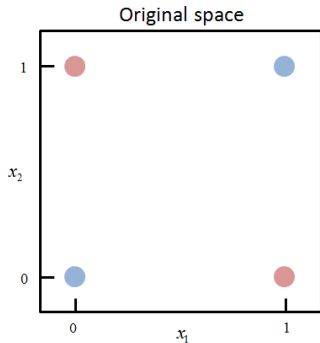
$$L(y, f(\mathbf{x})) = -(y \log f(\mathbf{x}) + (1 - y) \log(1 - f(\mathbf{x})))$$

A SINGLE NEURON: OPTIMIZATION / 2

- For a single neuron and both choices of τ the loss function is convex.
- The global optimum can be found with an iterative algorithm like gradient descent.
- A single neuron with logistic sigmoid function trained with the Bernoulli loss yields the same result as logistic regression when trained until convergence.
- Note: In the case of regression and the L2-loss, the solution can also be found analytically using the “normal equations”. However, in other cases a closed-form solution is usually not available.

Deep Learning

XOR-Problem



Learning goals

- Example problem a single neuron can not solve but a single hidden layer net can

EXAMPLE: XOR PROBLEM

- Suppose we have four data points

$$X = \{(0, 0)^T, (0, 1)^T, (1, 0)^T, (1, 1)^T\}$$

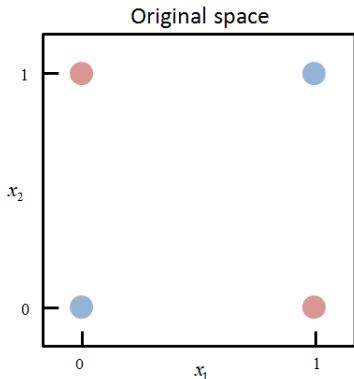
- The XOR gate (exclusive or) returns true, when an odd number of inputs are true:

x_1	x_2	XOR = y
0	0	0
0	1	1
1	0	1
1	1	0

- Can you learn the target function with a logistic regression model?

EXAMPLE: XOR PROBLEM / 2

- Logistic regression can not solve this problem. In fact, any model using simple hyperplanes for separation can not (including a single neuron).
- A small neural net can easily solve the problem by transforming the space!



EXAMPLE: XOR PROBLEM / 3

- Consider the following model:

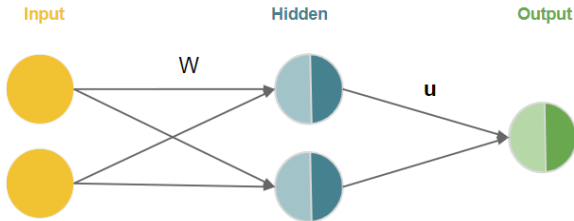


Figure: A neural network with two neurons in the hidden layer. The matrix W describes the mapping from x to z . The vector u from z to y .

EXAMPLE: XOR PROBLEM / 4

- Let use ReLU $\sigma(z) = \max\{0, z\}$ as activation function and a simple thresholding function $\tau(z) = [z > 0] = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{otherwise} \end{cases}$ as output transformation function. We can represent the architecture of the model by the following equation:

$$\begin{aligned} f(\mathbf{x} \mid \mathbf{W}, \mathbf{b}, \mathbf{u}, c) &= \tau\left(\mathbf{u}^\top \sigma(\mathbf{W}^\top \mathbf{x} + \mathbf{b}) + c\right) \\ &= \tau\left(\mathbf{u}^\top \max\{0, \mathbf{W}^\top \mathbf{x} + \mathbf{b}\} + c\right) \end{aligned}$$

- So how many parameters does our model have?
 - In a fully connected neural net, the number of connections between the nodes equals our parameters:

$$\underbrace{(2 \times 2)}_{\mathbf{W}} + \underbrace{(2 \times 1)}_{\mathbf{b}} + \underbrace{(2 \times 1)}_{\mathbf{u}} + \underbrace{(1)}_c = 9$$

EXAMPLE: XOR PROBLEM / 5

$$\text{Let } \mathbf{W} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \mathbf{b} = \begin{pmatrix} 0 \\ -1 \end{pmatrix}, \mathbf{u} = \begin{pmatrix} 1 \\ -2 \end{pmatrix}, c = -0.5$$

$$\mathbf{X} = \begin{pmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}, \mathbf{XW} = \begin{pmatrix} 0 & 0 \\ 1 & 1 \\ 1 & 1 \\ 2 & 2 \end{pmatrix}, \mathbf{XW} + \mathbf{B} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix}$$

Note: $\mathbf{X}$ is a $(n \times p)$ design matrix in which the *rows* correspond to the data points. $\mathbf{W}$, as usual, is a $(p \times m)$ matrix where each *column* corresponds to a single (hidden) neuron. $\mathbf{B}$ is a $(n \times m)$ matrix with $\mathbf{b}$ duplicated along the rows.

$$\mathbf{X} = \begin{pmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix} \quad \mathbf{W} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$$

EXAMPLE: XOR PROBLEM / 6

$$\text{Let } \mathbf{W} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \mathbf{b} = \begin{pmatrix} 0 \\ -1 \end{pmatrix}, \mathbf{u} = \begin{pmatrix} 1 \\ -2 \end{pmatrix}, c = -0.5$$

$$\mathbf{X} = \begin{pmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}, \mathbf{XW} = \begin{pmatrix} 0 & 0 \\ 1 & 1 \\ 1 & 1 \\ 2 & 2 \end{pmatrix}, \mathbf{XW} + \mathbf{B} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix}$$

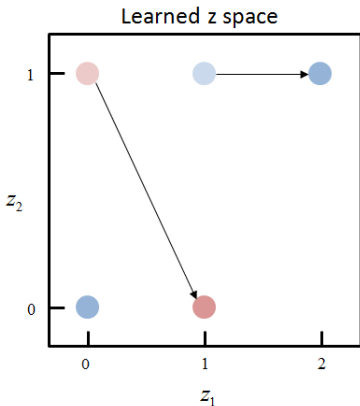
$$\mathbf{Z} = \max\{0, \mathbf{XW} + \mathbf{B}\} = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix}$$

- Note that we computed all examples at once.

EXAMPLE: XOR PROBLEM / 7

- The input points are mapped into transformed space to

$$\mathbf{Z} = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix}$$

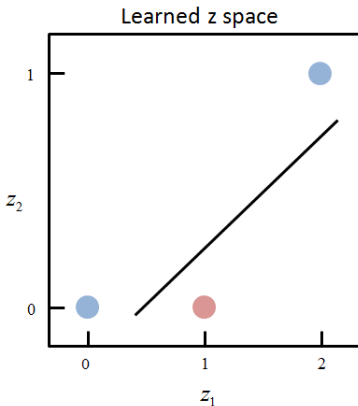


EXAMPLE: XOR PROBLEM / 8

- The input points are mapped into transformed space to

$$\mathbf{z} = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix}$$

which is easily separable.



EXAMPLE: XOR PROBLEM / 9

- In a final step we have to multiply the activated values of matrix **Z** with the vector **u** and add the bias term **c**:

$$f(\mathbf{x} \mid \mathbf{W}, \mathbf{b}, \mathbf{u}, c) = \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ -2 \end{pmatrix} + \begin{pmatrix} -0.5 \\ -0.5 \\ -0.5 \\ -0.5 \end{pmatrix} = \begin{pmatrix} -0.5 \\ 0.5 \\ 0.5 \\ -0.5 \end{pmatrix}$$

- And then apply the step function $\tau(z) = [z > 0]$. This solves the XOR problem perfectly!

x_1	x_2	XOR = y
0	0	0
0	1	1
1	0	1
1	1	0

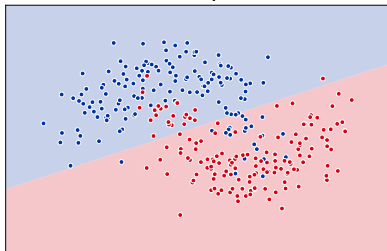
NEURAL NETWORKS : OPTIMIZATION

- In this simple example we actually “guessed” the values of the parameters for $\mathbf{W}$, $\mathbf{b}$, $\mathbf{u}$ and c .
- That won't work for more sophisticated problems!
- We will learn later about iterative optimization algorithms for automatically adapting weights and biases.
- An added complication is that the loss function is no longer convex. Therefore, there might not exist a single minimum.

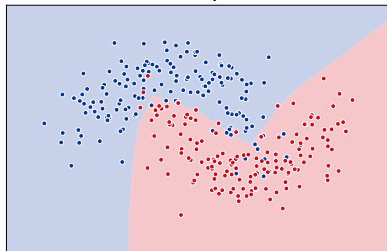
The same idea on real data (a preview)

XOR is four toy points. Here is the identical story on a real 2-D dataset – two interleaving “moons”:

Logistic regression: one straight cut
train accuracy = 0.84



MLP: a boundary that bends
train accuracy = 0.92



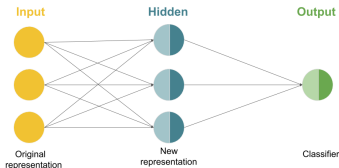
One neuron (logistic regression): one straight cut, stuck at ≈ 0.84 .

One hidden layer (32 ReLU): the boundary *bends* – ≈ 0.92 .

Exactly what the hidden layer did for XOR – learn a feature space where a straight boundary works – now on data that is not a toy. The mechanism is the very next slides.

Deep Learning

Single hidden layer neural networks



Learning goals

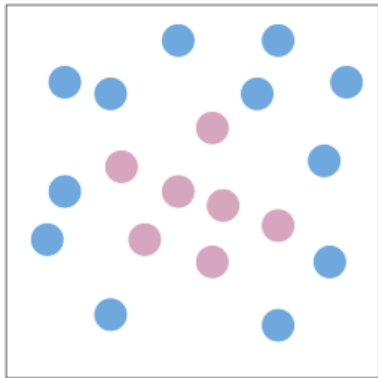
- Architecture of single hidden layer neural networks
- Representation learning/ understanding the advantage of hidden layers
- Typical (non-linear) activation functions

MOTIVATION

- The graphical way of representing simple functions/models, like logistic regression. Why is that useful?
- Because individual neurons can be used as building blocks of more complicated functions.
- Networks of neurons can represent extremely complex hypothesis spaces.
- Most importantly, it allows us to define the “right” kinds of hypothesis spaces to learn functions that are common in our universe in a data-efficient way (see Lin, Tegmark et al. 2016).

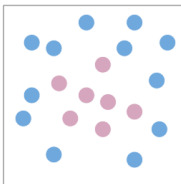
MOTIVATION / 2

Can a single neuron perform binary classification of these points?

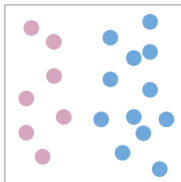


MOTIVATION / 3

- As a single neuron is restricted to learning only linear decision boundaries, its performance on the following task is quite poor:

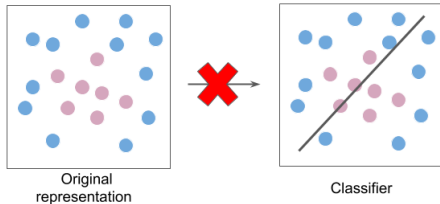


- However, the neuron can easily separate the classes if the original features are transformed (e.g., from Cartesian to polar coordinates):



MOTIVATION / 4

- Instead of classifying the data in the original representation,

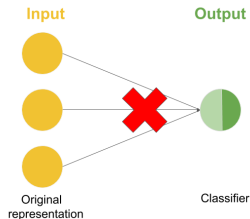


- we classify it in a new feature space.

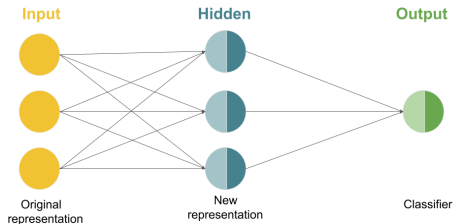


MOTIVATION / 5

- Analogously, instead of a single neuron,



- we use more complex networks.



REPRESENTATION LEARNING

- It is *very* critical to feed a classifier the “right” features in order for it to perform well.
- Before deep learning took off, features for tasks like machine vision and speech recognition were “hand-designed” by domain experts. This step of the machine learning pipeline is called **feature engineering**.
- DL automates feature engineering. This is called **representation learning**.

SINGLE HIDDEN LAYER NETWORKS

Single neurons perform a 2-step computation:

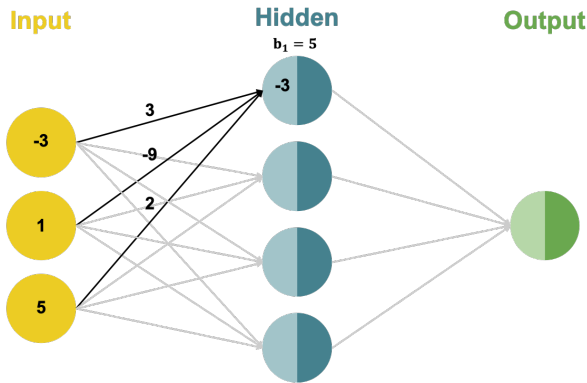
- ➊ **Affine Transformation:** a weighted sum of inputs plus bias.
- ➋ **Activation:** a non-linear transformation on the weighted sum.

Single hidden layer networks consist of two layers (without input layer):

- ➊ **Hidden Layer:** having a set of neurons.
 - ➋ **Output Layer:** having one or more output neurons.
- Multiple inputs are simultaneously fed to the network.
 - Each neuron in the hidden layer performs a 2-step computation.
 - The final output of the network is then calculated by another 2-step computation performed by the neuron in the output layer.

SINGLE HIDDEN LAYER NETWORKS: EXAMPLE

Each neuron in the hidden layer performs an **affine transformation** on the inputs:

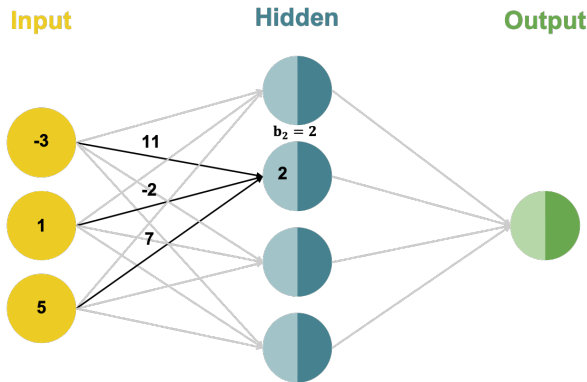


$$Z_{1,in} = w_{11}x_1 + w_{21}x_2 + w_{31}x_3 + b_1$$

$$Z_{1,in} = 3 * (-3) + (-9) * 1 + 2 * 5 + 5 = -3$$

SINGLE HIDDEN LAYER NETWORKS: EXAMPLE

Each neuron in the hidden layer performs an **affine transformation** on the inputs:

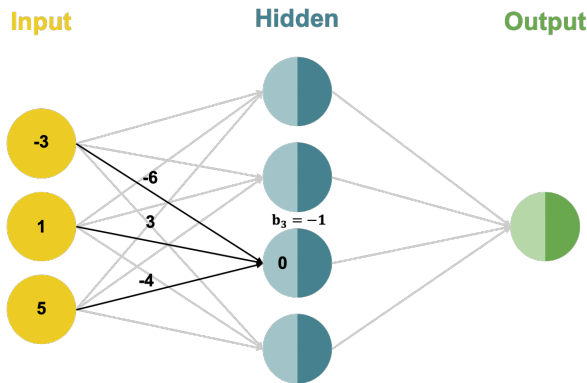


$$z_{2,\text{in}} = w_{12}x_1 + w_{22}x_2 + w_{32}x_3 + b_2$$

$$z_{2,\text{in}} = 11 * (-3) + (-2) * 1 + 7 * 5 + 2 = 2$$

SINGLE HIDDEN LAYER NETWORKS: EXAMPLE

Each neuron in the hidden layer performs an **affine transformation** on the inputs:

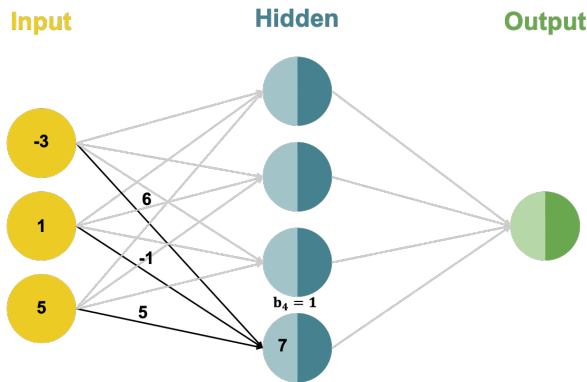


$$Z_{3,in} = w_{13}x_1 + w_{23}x_2 + w_{33}x_3 + b_3$$

$$Z_{3,in} = (-6) * (-3) + 3 * 1 + (-4) * 5 - 1 = 0$$

SINGLE HIDDEN LAYER NETWORKS: EXAMPLE

Each neuron in the hidden layer performs an **affine transformation** on the inputs:

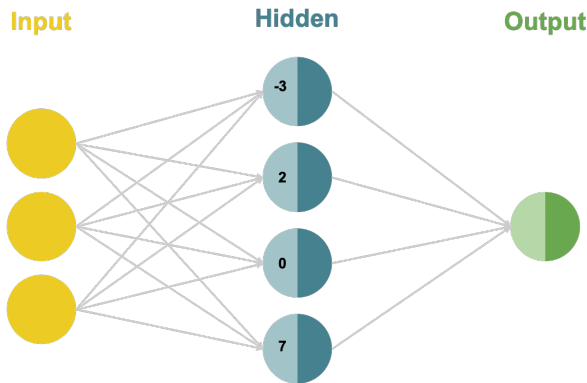


$$Z_{4,\text{in}} = w_{14}x_1 + w_{24}x_2 + w_{34}x_3 + b_4$$

$$Z_{4,\text{in}} = 6 * (-3) + (-1) * 1 + 5 * 5 + 1 = 7$$

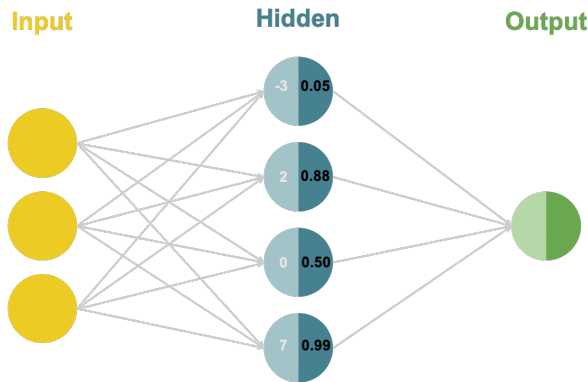
SINGLE HIDDEN LAYER NETWORKS: EXAMPLE

Each neuron in the hidden layer performs an **affine transformation** on the inputs:



SINGLE HIDDEN LAYER NETWORKS: EXAMPLE

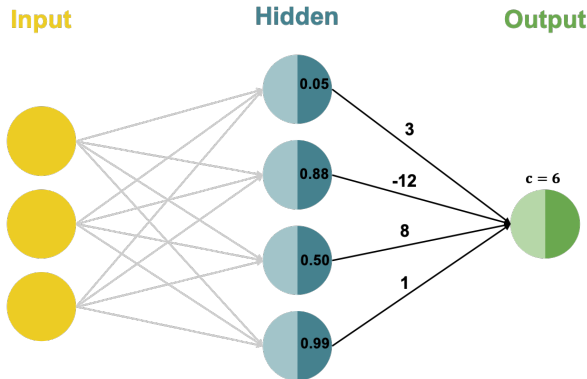
Each hidden neuron performs a non-linear **activation** transformation on the weight sum:



$$z_{i,\text{out}} = \sigma(z_{i,\text{in}}) = \frac{1}{1 + e^{-z_{i,\text{in}}^{(i)}}}$$

SINGLE HIDDEN LAYER NETWORKS: EXAMPLE

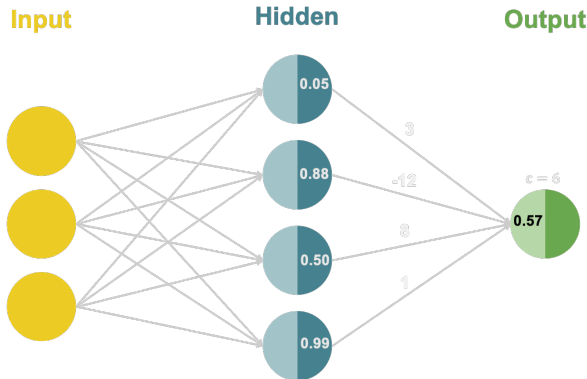
The output neuron performs an **affine transformation** on its inputs:



$$f_{\text{in}} = u_1 z_{1,\text{out}} + u_2 z_{2,\text{out}} + u_3 z_{3,\text{out}} + u_4 z_{4,\text{out}} + c$$

SINGLE HIDDEN LAYER NETWORKS: EXAMPLE

The output neuron performs an **affine transformation** on its inputs:

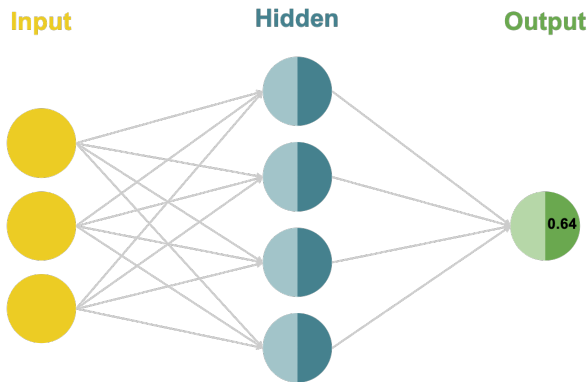


$$f_{in} = u_1 z_{1,out} + u_2 z_{2,out} + u_3 z_{3,out} + u_4 z_{4,out} + c$$

$$f_{in} = 3 * 0.05 + (-12) * 0.88 + 8 * 0.50 + 1 * 0.99 + 6 = 0.57$$

SINGLE HIDDEN LAYER NETWORKS: EXAMPLE

The output neuron performs a non-linear **activation** transformation on the weight sum:



$$f_{\text{out}} = \sigma(f_{\text{in}}) = \frac{1}{1 + e^{-f_{\text{in}}}}$$
$$f_{\text{out}} = \frac{1}{1 + e^{-0.57}} = 0.64$$

HIDDEN LAYER: ACTIVATION FUNCTION

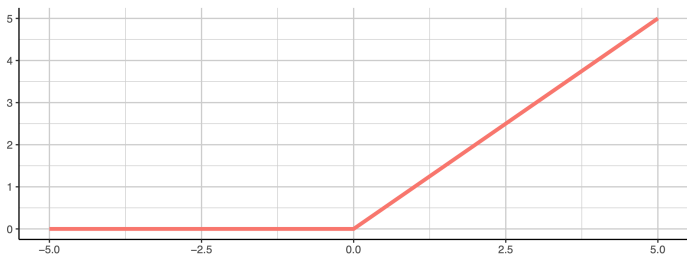
- If the hidden layer does not have a non-linear activation, the network can only learn linear decision boundaries.
- A lot of different activation functions exist.

HIDDEN LAYER: ACTIVATION FUNCTION

ReLU Activation:

- Currently the most popular choice is the ReLU (rectified linear unit):

$$\sigma(v) = \max(0, v)$$

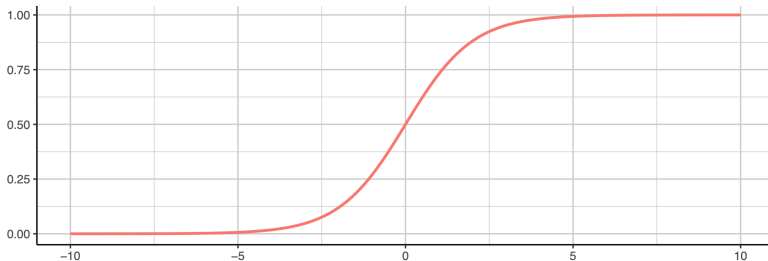


HIDDEN LAYER: ACTIVATION FUNCTION / 2

Sigmoid Activation Function:

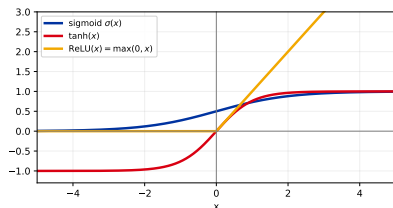
- The sigmoid function can be used even in the hidden layer:

$$\sigma(v) = \frac{1}{1 + \exp(-v)}$$

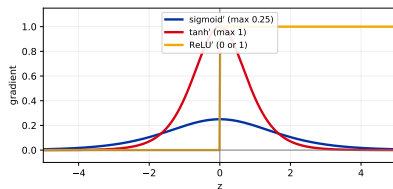


Activation functions, side by side

The last slides showed ReLU and sigmoid one at a time. Here they are together – plus **tanh** – and, more importantly, their **gradients**:



the functions – all add “bend”



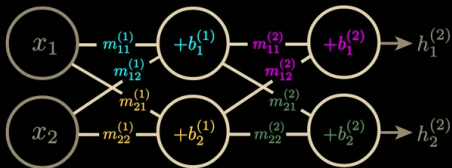
the derivatives – how much signal each passes back

Training moves weights along *gradients* (next chunk). **Sigmoid's derivative peaks at only 0.25** and dies at both ends; **ReLU's is exactly 1** for $z > 0$ – which is why ReLU is the default for hidden layers.

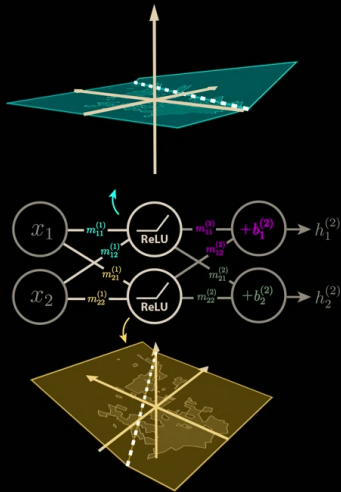
$$h_1^{(2)} = m_{11}^{(2)} h_1^{(1)} + m_{12}^{(2)} h_2^{(1)} + b_1^{(2)}$$

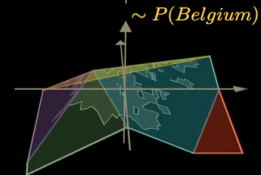
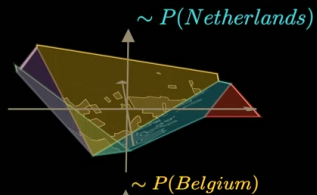
$$\Rightarrow h_1^{(2)} = m_{11}^{(2)} \left(m_{11}^{(1)} x_1 + m_{12}^{(1)} x_2 + b_1^{(1)} \right) + m_{12}^{(2)} \left(m_{21}^{(1)} x_1 + m_{22}^{(1)} x_2 + b_2^{(1)} \right) + b_1^{(2)}$$

$$h_1^{(1)} = m_{11}^{(1)} x_1 + m_{12}^{(1)} x_2 + b_1^{(1)}$$



$$h_2^{(1)} = m_{21}^{(1)} x_1 + m_{22}^{(1)} x_2 + b_2^{(1)}$$





Now make it yours

The fastest way to understand these slides is to watch a network learn – and then to see what question comes next.

See it learn: play with it

The single best way to internalise this chunk is to **watch the boundary bend** live:

- ▶ **TensorFlow Playground** – playground.tensorflow.org. Pick the *XOR* or *two-spirals* data. Start with **0 hidden layers** (it fails, like a single neuron), then add **one hidden layer** and watch it become separable. Toggle **ReLU vs sigmoid** and add neurons.
- ▶ **3Blue1Brown** – “**Neural networks**” (chapters 1–4). The clearest visual intuition for what the weights are doing.
- ▶ mlu-explain.github.io/neural-networks – a short, clickable explainer.
- ▶ **Karpathy, “Neural Networks: Zero to Hero”** (micrograd). Builds a neuron – and the training that comes next – from scratch in Python.

Try to **predict** before you press play: how many hidden neurons does XOR need? What happens with *zero* hidden layers? Twenty minutes here beats an hour of slides.

Recap: from a neuron to a network

- ▶ A **neuron** = affine map $\mathbf{w}^\top \mathbf{x} + b$, then an activation τ . Choose τ and you recover a model you know: identity $\rightarrow$ linear regression, sigmoid $\rightarrow$ logistic regression.
- ▶ **One neuron draws one straight boundary.** That is why it cannot solve XOR, or any curved problem.
- ▶ A **hidden layer learns a new feature space** – it re-coordinatises the data (XOR's points collapsed so a straight cut works). This is **representation learning**.
- ▶ More hidden neurons / layers = richer feature spaces = more flexible boundaries.

Everything here is still “a stack of affine maps and nonlinearities” – exactly the idea behind the huge models in the intro deck, just tiny.

The open question: who picks the weights?

In the XOR and hidden-layer examples, the weights were **guessed** for us. That will not scale:

- ▶ real networks have **millions** of weights – nobody sets them by hand;
- ▶ once there is a hidden layer the loss is **no longer convex**, so there is no neat closed-form solution and no single guaranteed minimum.

So how do we *learn* good weights automatically?

The answer is training. **Gradient descent** moves the weights downhill and **backpropagation** gets every gradient in one pass – the algorithm that makes deep learning possible. We get there later in the chapter, after first generalizing the architecture (deeper nets, many classes).